

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)

КУРСОВОЙ ПРОЕКТ

по курсу
«Разработка веб-приложений»

ТЕМА

«Веб-платформа для поиска собеседников и языковых клубов
для разговорной практики английского языка»

Выполнил: Стальмахов Иван

Группа: 241-3211

Проверил: Кружалов Алексей Сергеевич

Москва, 2026

**«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)**

УТВЕРЖДАЮ
заведующая кафедрой
«Инфокогнитивные технологии»

_____ / Е. А. Пухова /

«____» _____ 2026 г.

ЗАДАНИЕ

на выполнение курсовой работы (проекта)

Стальмахову Ивану Сергеевичу,

обучающемуся группы 241-3211,

направления подготовки 09.03.01 «Информатика и вычислительная техника»

по дисциплине «Разработка веб-приложений»

на тему «Разработка веб-приложения для поиска собеседников и языковых клубов для разговорной практики английского языка с личными кабинетами пользователей»

1. Исходные данные к работе (проекту): информационные ресурсы в сети интернет, научные публикации в открытой печати.

2. Содержание задания по курсовой работе (проекту) – перечень вопросов, подлежащих разработке:

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Раздел 1. Анализ предметной области			
Задача 1.1. Обзор существующих программных продуктов по теме работы	10	03.04.2026	
Задача 1.2. Анализ программных инструментов разработки веб-приложений	7	06.04.2026	
Задача 1.3. Формулировка цели и задач работы	8	11.04.2026	
Раздел 2. Проектирование веб-приложения			
Задача 2.1. Анализ целевой аудитории	5	14.04.2026	

Разрабатываемый вопрос	Объем, %	Срок	Примечание
Задача 2.2. Описание функциональности приложения (диаграмма вариантов использования, user story)	7	18.04.2026	
Задача 2.3. Проектирование модели данных (ER-диаграмма, логическая и физическая схемы БД)	8	23.04.2026	
Задача 2.4. Разработка макетов страниц (Wireframe)	5	27.04.2026	
Раздел 3. Разработка веб-приложения			
Задача 3.1. Разработка базовой структуры приложения и вёрстка шаблонов страниц	8	01.05.2026	
Задача 3.2. Реализация аутентификации пользователей	7	05.05.2026	
Задача 3.3. Реализация CRUD-интерфейса для пользователей, карточек клубов и встреч (событий)	7	09.05.2026	
Задача 3.4. Реализация системы поиска, фильтрации и записи на встречи	12	14.05.2026	
Задача 3.5. Разработка системы тем, интересов и уровней владения языком	6	18.05.2026	
Раздел 4. Оформление итогов работы			
Задача 4.1. Создание Git-репозитория с кодом проекта	3	21.05.2026	
Задача 4.2. Деплой приложения на хостинг	4	24.05.2026	
Задача 4.3. Оформление отчёта о проделанной работе	3	26.05.2026	

Руководитель курсовой работы (проекта):

старший преподаватель кафедры «Инфокогнитивные технологии»

«____» _____ 2026 г.

(подпись)

А. С. Кружалов

Дата выдачи задания

«31» марта 2026 г.

Дата сдачи выполненной работы

«____» _____ 2026 г.

Задание принял к исполнению

«31» марта 2026 г.

(подпись)

И.С. Стальмахов

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	5
ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ	6
1.1 Обзор существующих программных продуктов	6
1.2 Анализ программных инструментов разработки	7
1.3 Формулировка цели и задач работы	10
ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ	11
2.1 Анализ целевой аудитории	11
2.2 Описание функциональности приложения	11
2.3 Проектирование модели данных	13
2.4 Разработка макетов страниц	15
ГЛАВА 3. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ	20
3.1 Общая архитектура приложения	20
3.2 Реализация модели данных	20
3.3 Защита данных средствами Row Level Security	21
3.4 Аутентификация и роли пользователей	22
3.5 Сервисный слой и серверные действия	23
3.6 Реализация ключевых модулей	24
3.6.1 Каталог встреч и фильтрация	24
3.6.2 Запись на встречу и защита от перепревышения	25
3.6.3 Управление клубами и встречами	26
3.6.4 Загрузка аватара	28
3.6.5 Выгрузка списка участников в CSV	28
3.6.6 Административная панель	29
3.7 Тестирование	29
3.8 Деплой приложения	30
ЗАКЛЮЧЕНИЕ	32

ВВЕДЕНИЕ

Актуальность темы. Разговорная практика является ключевым, но наиболее труднодоступным компонентом изучения иностранного языка. Аудиторных занятий в университете недостаточно для формирования навыка свободной речи, а самостоятельный поиск собеседника или языкового клуба затруднён из-за рассредоточенности информации по чатам и социальным сетям. Разработка специализированной веб-платформы, объединяющей каталог языковых клубов и инструменты записи на встречи, позволяет устранить этот барьер.

Степень разработанности проблемы. Существующие решения – приложения для поиска собеседника (HelloTalk, Tandem) и универсальные платформы мероприятий (Meetup) – решают задачу частично: первые не поддерживают групповой формат, вторые не имеют языковой специализации. Специализированного веб-сервиса, сочетающего клубный формат, уровни владения языком и инструменты записи на встречи, на рынке не представлено.

Объект исследования – процессы организации разговорной практики английского языка в формате языковых клубов.

Предмет исследования – программные средства для автоматизации поиска языковых клубов и управления записью участников на встречи в рамках веб-платформы.

Целью данной работы является проектирование и разработка веб-приложения для поиска языковых клубов и записи на разговорные встречи по английскому языку с разграничением доступа по ролям пользователей.

ГЛАВА 1. АНАЛИЗ ПРЕДМЕТНОЙ ОБЛАСТИ

1.1 Обзор существующих программных продуктов

На рынке представлено несколько решений, частично закрывающих потребность в разговорной практике иностранного языка. Для структурированного сравнения выделены пять критериев, определяющих пригодность платформы для организации разговорной языковой практики:

- **Групповой формат встреч** – разговорная практика эффективнее в малой группе, чем в формате один на один: участники слышат разные акценты и стили речи, а нагрузка на каждого ниже.
- **Запись участников на встречи** – необходима для планирования мероприятий, контроля числа участников и уведомления организатора.
- **Уровень языка (CEFR) в профиле** – международная шкала CEFR позволяет однозначно указать уровень владения языком и подобрать встречу подходящей сложности.
- **Фильтрация встреч по уровню языка** – без фильтрации участник вынужден просматривать все встречи подряд; фильтр по CEFR сокращает время поиска и снижает вероятность записи на несоответствующее мероприятие.
- **Языковая специализация** – платформа, заточенная под языковую практику, может предоставлять релевантные инструменты: профиль с уровнем языка, тематические клубы, – чего не даёт универсальный сервис мероприятий.

HelloTalk – мобильное приложение для поиска собеседника по переписке и голосовым звонкам. Поддерживает уровни языка и интересы в профиле, однако ориентировано исключительно на общение один на один. Групповой формат и запись на встречи не предусмотрены.

Tandem – аналогичное приложение для языкового обмена. Основная механика – найти носителя языка и переписываться. Клубы и расписание встреч отсутствуют.

Meetup – платформа для организации мероприятий любой тематики. Поддерживает создание групп, расписание и запись участников, однако не имеет языковой специализации: в профиле отсутствуют уровни CEFR, фильтрация по уровню владения языком не реализована.

Сравнительный анализ аналогов по выделенным критериям представлен в таблице 1.1.

Таблица 1.1 – Сравнение существующих решений

Критерий	HelloTalk	Tandem	Meetup	Наша платформа
Групповой формат встреч	–	–	+	+
Запись участников на встречи	–	–	+	+
Уровень языка (CEFR) в профиле	+	+	–	+
Фильтрация встреч по уровню языка	–	–	–	+
Языковая специализация	+	+	–	+

Анализ показал, что ни одно из рассмотренных решений не сочетает языковую специализацию и клубный формат с записью на конкретные встречи, что подтверждает необходимость разработки специализированной платформы.

1.2 Анализ программных инструментов разработки

Разработка современного веб-приложения требует выбора инструментов в четырёх ключевых категориях: fullstack-фреймворк, платформа данных и аутентификации, CSS-фреймворк и хостинг. В каждой категории существует несколько распространённых решений; ниже рассмотрены доступные варианты и обоснован итоговый выбор.

Fullstack-фреймворк. Среди актуальных решений выделяются Next.js, Nuxt.js и SvelteKit. Nuxt.js и SvelteKit ориентированы на экосистемы Vue и Svelte соответственно, тогда как Next.js построен на React – наиболее распространённой библиотеке для построения пользовательских интерфейсов. Next.js 15 предоставляет встроенный серверный слой (App Router), что позволяет реализовать клиентскую и серверную части в едином репозитории без отдельного backend-сервера.

Платформа данных и аутентификации. Рассмотрены три варианта:

Firebase (Google), Supabase и самостоятельно развёрнутый PostgreSQL. Firebase использует документно-ориентированную модель хранения данных, что усложняет реализацию реляционных связей между сущностями. Самостоятельный PostgreSQL требует отдельной реализации аутентификации, управления сессиями и хранилища файлов. Supabase предоставляет реляционную базу данных PostgreSQL, встроенную систему аутентификации и файловое хранилище в рамках единой облачной платформы, что сокращает объём инфраструктурного кода.

CSS-фреймворк. Среди распространённых вариантов – Bootstrap, Tailwind CSS и CSS Modules. Bootstrap предлагает готовые компоненты с фиксированным визуальным стилем, что ограничивает гибкость оформления. CSS Modules требуют ручного написания всех стилей. Tailwind CSS реализует утилитарный подход: стили задаются классами прямо в разметке, что ускоряет разработку интерфейса без создания отдельных таблиц стилей.

Хостинг. Рассмотрены Vercel, Netlify и VPS с ручной настройкой (nginx, SSL, CI/CD). VPS предоставляет максимальный контроль, но требует значительных затрат на настройку инфраструктуры. Netlify ориентирован преимущественно на статические сайты. Vercel разработан той же командой, что и Next.js, обеспечивает автоматический деплой при обновлении репозитория и предоставляется бесплатно для учебных проектов.

Сравнение вариантов по каждой категории представлено в таблицах 1.2–1.5.

Таблица 1.2 – Сравнение fullstack-фреймворков

Критерий	Next.js	Nuxt.js	SvelteKit
Серверный рендеринг (SSR)	+	+	+
Встроенный API без отдельного сервера	+	+	+
Экосистема React	+	–	–
Широкое распространение в индустрии	+	–	–

Таблица 1.3 – Сравнение платформ данных и аутентификации

Критерий	Firestore	Supabase	PostgreSQL
Реляционная модель данных	–	+	+
Встроенная аутентификация	+	+	–
Файловое хранилище	+	+	–
Открытый исходный код	–	+	+

Таблица 1.4 – Сравнение CSS-фреймворков

Критерий	Bootstrap	Tailwind CSS	CSS Modules
Гибкость оформления	–	+	+
Скорость разработки интерфейса	+	+	–
Отсутствие избыточных стилей	–	+	+

Таблица 1.5 – Сравнение платформ хостинга

Критерий	Vercel	Netlify	VPS
Автодеплой из Git	+	+	–
Нативная поддержка Next.js	+	–	–
Бесплатный тариф	+	+	–
Не требует настройки сервера	+	+	–

Таким образом, для реализации проекта выбраны Next.js 15, Supabase, Tailwind CSS и Vercel – совокупность инструментов, обеспечивающая fullstack-разработку в едином репозитории, встроенную аутентификацию и хранилище данных, гибкую стилизацию и автоматический деплой при минимальных инфраструктурных затратах.

1.3 Формулировка цели и задач работы

На основе проведённого анализа существующих продуктов и инструментов разработки сформулирована цель исследования.

Цель работы – разработка веб-приложения для поиска языковых клубов и записи на разговорные встречи по английскому языку.

Для достижения поставленной цели необходимо решить следующие **задачи**:

1. Провести анализ существующих программных продуктов и инструментов разработки;
2. Спроектировать архитектуру приложения и структуру базы данных;
3. Реализовать систему аутентификации и разграничения доступа по ролям;
4. Реализовать CRUD-интерфейс для управления клубами и встречами;
5. Реализовать запись участников на встречи и фильтрацию по уровню языка и дате;
6. Обеспечить загрузку аватара и выгрузку списка участников в формате CSV;
7. Провести тестирование и развернуть приложение на хостинге.

ГЛАВА 2. ПРОЕКТИРОВАНИЕ ВЕБ-ПРИЛОЖЕНИЯ

2.1 Анализ целевой аудитории

Разрабатываемая платформа ориентирована на людей, изучающих английский язык и заинтересованных в разговорной практике в групповом формате. В системе выделяются три роли пользователей.

Участник – любой зарегистрированный пользователь. Ведёт профиль с уровнем языка (CEFR), просматривает каталог клубов, записывается на встречи. Может в любой момент создать клуб, став его организатором.

Организатор – пользователь, создавший один или несколько клубов. Управляет расписанием встреч, просматривает список записавшихся участников и выгружает его в CSV для подготовки к занятию.

Администратор – привилегированный пользователь, ответственный за модерацию платформы. Блокирует нарушителей, скрывает неактивные клубы и имеет доступ ко всем данным системы.

Для каждой роли составлен профиль пользователя (Persona), отражающий типичный сценарий использования платформы (таблица 2.1).

Таблица 2.1 – Профили пользователей (Personas)

Роль	Цель	Типичное действие
Участник	Найти встречу своего уровня и записаться	Открыть каталог, выставить фильтр по уровню CEFR, выбрать встречу, нажать «Записаться»
Организатор	Набрать группу и провести встречу	Создать клуб, добавить встречу, скачать CSV со списком участников
Администратор	Поддерживать качество контента	Просмотреть жалобы, заблокировать нарушителя, скрыть неактивный клуб

2.2 Описание функциональности приложения

Функциональные возможности системы определяются через взаимодействие пользователей трёх ролей с платформой. Участник просматривает каталог клубов, фильтрует встречи по уровню языка и дате, записывается на встречи и управляет своим профилем. Организатор, являясь участником,

дополнительно создаёт клубы и встречи, отслеживает наполнение группы и выгружает список участников в CSV-файл. Администратор модерирует пользователей и клубы платформы.

На рисунке 2.1 представлена диаграмма вариантов использования системы.

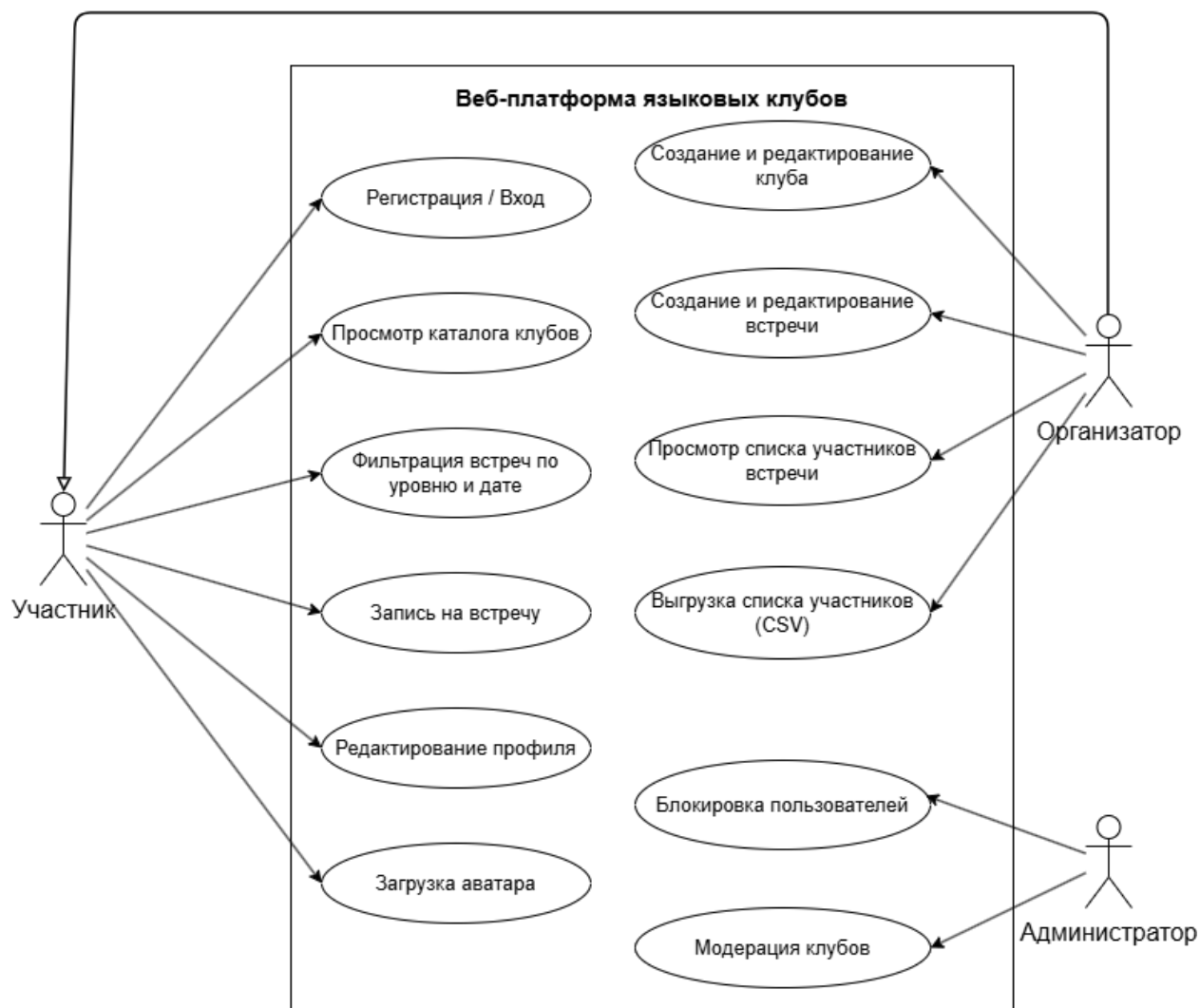


Рисунок 2.1 – Диаграмма вариантов использования системы

2.3 Проектирование модели данных

На рисунке 2.2 представлена ER-диаграмма базы данных.

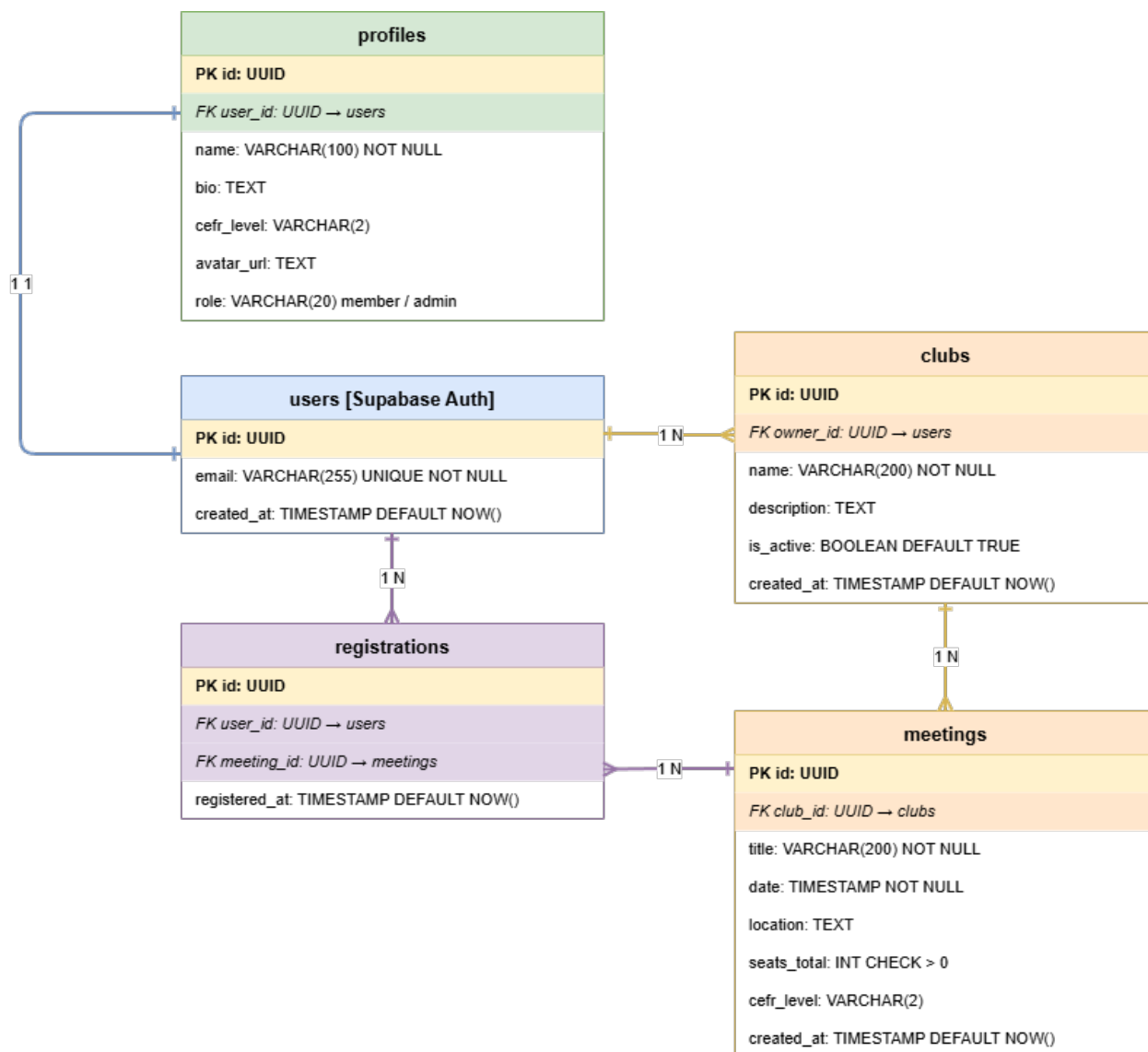


Рисунок 2.2 – ER-диаграмма базы данных

База данных реализована на PostgreSQL в составе платформы Supabase. Логическая схема включает пять таблиц, структура которых представлена ниже.

Таблица 2.2 – Структура таблицы users (управляется Supabase Auth)

Поле	Тип	Ограничение	Описание
id	UUID	PRIMARY KEY	Уникальный идентификатор
email	VARCHAR(255)	UNIQUE, NOT NULL	Адрес электронной почты
created_at	TIMESTAMP	DEFAULT NOW()	Дата регистрации

Таблица 2.3 – Структура таблицы profiles

Поле	Тип	Ограничение	Описание
id	UUID	PRIMARY KEY	Уникальный идентификатор
user_id	UUID	FOREIGN KEY	Ссылка на таблицу users
name	VARCHAR(100)	NOT NULL	Отображаемое имя
bio	TEXT	NULL	Описание «о себе»
cefr_level	VARCHAR(2)	NULL	Уровень языка: A1–C2
avatar_url	TEXT	NULL	URL аватара в хранилище
role	VARCHAR(20)	DEFAULT 'member'	Роль: member / admin

Таблица 2.4 – Структура таблицы clubs

Поле	Тип	Ограничение	Описание
id	UUID	PRIMARY KEY	Уникальный идентификатор
owner_id	UUID	FOREIGN KEY	Создатель клуба (ссылка на users)
name	VARCHAR(200)	NOT NULL	Название клуба
description	TEXT	NULL	Описание клуба
is_active	BOOLEAN	DEFAULT TRUE	Признак активности
created_at	TIMESTAMP	DEFAULT NOW()	Дата создания

Таблица 2.5 – Структура таблицы meetings

Поле	Тип	Ограничение	Описание
id	UUID	PRIMARY KEY	Уникальный идентификатор
club_id	UUID	FOREIGN KEY	Клуб встречи (ссылка на clubs)
title	VARCHAR(200)	NOT NULL	Тема встречи
date	TIMESTAMP	NOT NULL	Дата и время начала
location	TEXT	NULL	Адрес проведения
seats_total	INT	CHECK > 0	Максимальное число мест
cefr_level	VARCHAR(2)	NULL	Целевой уровень участников
created_at	TIMESTAMP	DEFAULT NOW()	Дата создания записи

Таблица 2.6 – Структура таблицы registrations

Поле	Тип	Ограничение	Описание
id	UUID	PRIMARY KEY	Уникальный идентификатор
user_id	UUID	FOREIGN KEY	Участник (ссылка на users)
meeting_id	UUID	FOREIGN KEY	Встреча (ссылка на meetings)
registered_at	TIMESTAMP	DEFAULT NOW()	Дата и время записи

Уникальное ограничение `UNIQUE(user_id, meeting_id)` предотвращает повторную запись одного участника на ту же встречу. Счётчик занятых мест вычисляется агрегатным запросом `COUNT(*)` к таблице `registrations` по идентификатору встречи.

2.4 Разработка макетов страниц

Для ключевых страниц разработаны wireframe-макеты, отражающие расположение функциональных элементов интерфейса.

поиск по теме встречи или клубу...

Войти

Регистрация

Фильтры

УРОВЕНЬ CEFR

A1

A2

B1

B2

C1

C2

Дата от

ДД.ММ.ГГГГ

Дата до

ДД.ММ.ГГГГ

Сбросить фильтры

Каталог встреч

Найдено 4 встречи по фильтрам

Сортировка: По дате

Обсуждение фильма «Inception»

B2

7 мая, 19:00–20:30

Адрес: Москва, Кофейня Surf, ул. Тверская 12

Обсудим сюжет и идеи фильма Кристофера Нолана. Практикуем аргументацию и описание эмоций. Посмотрите фильм заранее — подготовка не нужна.

КЛУБ

Организатор

Film Discussion Club

Дмитрий Р.

Записалось 8 из 12

Свободно: 4

Карточка встречи →

AI & The Future of Work

C1

9 мая, 18:00–19:30

Адрес: Москва, Коворкинг Hub, Садовая 5

Дискуссия об искусственном интеллекте и рынке труда. Формат дебатов: каждый защищает свою позицию, затем открытое обсуждение.

КЛУБ

Организатор

Advanced Debates

Елена В.

Записалось 9 из 10

Осталось 1!

Карточка встречи →

Travel Stories: My Best Trip

B1

10 мая, 11:00–12:00

Адрес: Москва, Библиотека им. Ленина, зал 3

Расскажите о своём лучшем путешествии на английском. Неформальный формат, все уровни в рамках B1 приветствуются.

КЛУБ

Организатор

Morning Talks Club

Михаил С.

Записалось 4 из 15

Свободно: 11

Карточка встречи →

Job Interviews in English

B2

12 мая, 19:30–21:00

Адрес: Москва, БЦ Романов, переговорная 2

Разбираем типичные вопросы на собеседовании, отработываем ответы в парах. Полезно всем, кто ищет работу в международной компании.

КЛУБ

Организатор

Business English Lab

Анна К.

Записалось 2 из 10

Свободно: 8

Карточка встречи →

1

16

Карточка встречи (рисунок 2.4) – детальная страница отдельной встречи. Отображает тему, дату, адрес, клуб, организатора, описание и уровень языка. В правой части расположен блок записи со счётчиком «X из N мест занято» и кнопкой «Записаться»; для уже записавшихся участников кнопка меняется на «Отменить запись».

The mockup shows a web interface for a meeting card. At the top, there are navigation buttons 'Каталог встреч' and 'Мои встречи', and a user profile 'АК Алёна К.'. The main title is 'Обсуждение фильма «Inception»' with a 'B2' level tag. The meeting details are organized into a grid:

- Дата и время:** 7 мая 2026, 19:00–20:30
- Адрес:** Москва, Кофейня Surf, ул. Тверская 12
- Клуб:** Film Discussion Club
- Организатор:** Дмитрий Романов

Below the grid is a description box titled 'ОПИСАНИЕ ВСТРЕЧИ' containing text about discussing the movie 'Inception' and practicing English conversation. On the right side, there is a 'Запись на встречу' section showing '8 из 12 мест занято' with a progress bar and a 'Записаться' button. Below the button, it says '← не записан'.

Рисунок 2.4 – Макет страницы карточки встречи

Профиль участника (рисунок 2.5) – личная страница пользователя. В левой колонке – аватар, имя, бейдж уровня CEFR и раздел «о себе». В правой колонке – форма редактирования профиля с загрузкой аватара, полями имени и био, выбором уровня языка.

Каталог встреч

Мои встречи

АК Алёна К. ▾

АК

Алёна Кузнецова

В2

Изучаю английский 3 года,
люблю обсуждать технологии и
путешествия

Редактировать профиль

ФОТО ПРОФИЛЯ

АК

перетащите файл или нажмите для загрузки

ИМЯ

Алёна Кузнецова

О СЕБЕ (BIO)

Изучаю английский 3 года, люблю обсуждать технологии и путешествия

УРОВЕНЬ ЯЗЫКА

A1

A2

B1

B2

C1

C2

Сохранить изменения

Отмена

Рисунок 2.5 – Макет страницы профиля участника

Панель организатора (рисунок 2.6) – страница управления клубом. Содержит название и описание клуба, фильтр встреч (все / предстоящие / прошедшие), список встреч с прогресс-баром заполненности и кнопками «Редактировать», «Удалить», «Скачать CSV», а также кнопку создания новой встречи.

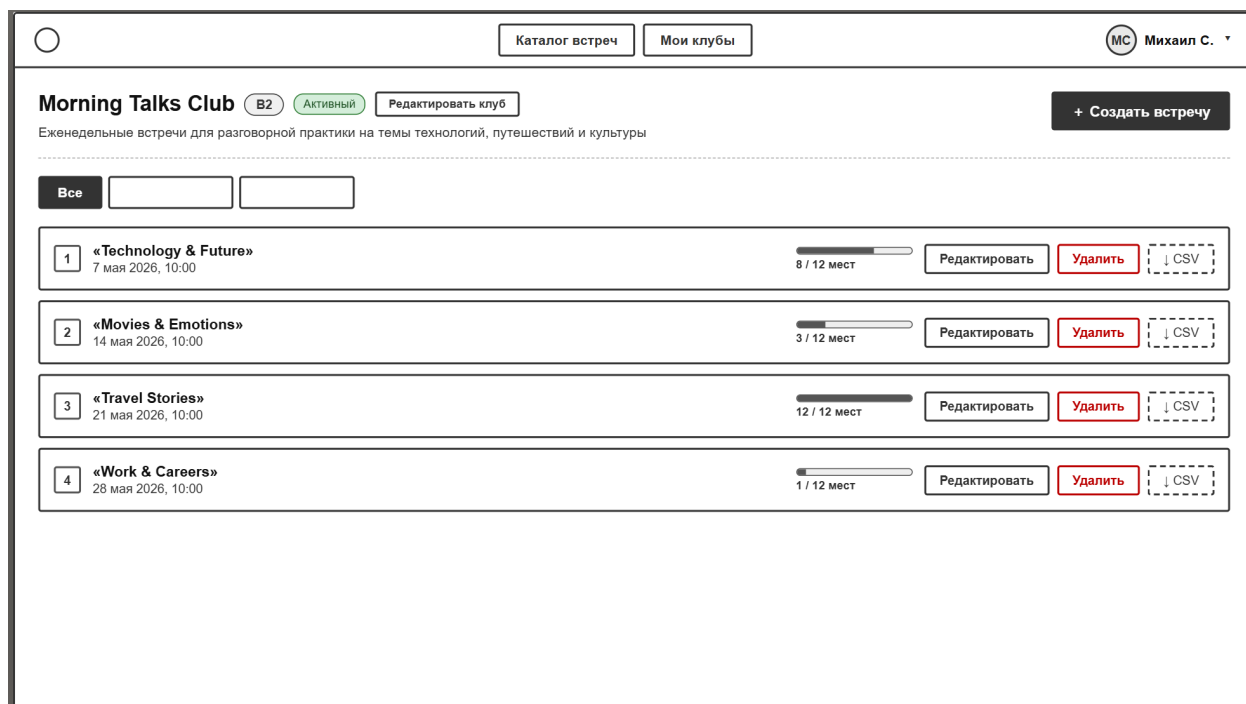


Рисунок 2.6 – Макет панели организатора

ГЛАВА 3. РАЗРАБОТКА ВЕБ-ПРИЛОЖЕНИЯ

3.1 Общая архитектура приложения

Приложение реализовано как fullstack-проект на платформе Next.js 15 с использованием парадигмы App Router. Слой представления и серверной логики размещён в едином репозитории, а в роли источника данных, системы аутентификации и файлового хранилища выступает облачная платформа Supabase.

Архитектурно приложение разделено на четыре слоя.

- **Серверные компоненты (Server Components)** – рендерятся на сервере, выполняют запросы к базе данных и формируют HTML до отправки клиенту. Используются для всех страниц по умолчанию.
- **Клиентские компоненты (Client Components)** – помеченные директивой `'use client'`. Применяются только там, где необходимо состояние и обработка событий: формы, переключатели, фильтры, кнопка записи на встречу.
- **Сервисный слой (lib/services/)** – инкапсулирует все обращения к Supabase. Компоненты никогда не обращаются к базе данных напрямую, что обеспечивает единое место для логики доступа к данным и упрощает тестирование.
- **Серверные действия (Server Actions)** – функции с директивой `'use server'`, используемые для мутаций (создание, обновление, удаление). Принимают FormData и вызываются непосредственно из форм без отдельного API-маршрута.

Структура каталогов проекта представлена в таблице 3.1.

3.2 Реализация модели данных

Схема базы данных, спроектированная в главе 2, реализована в PostgreSQL средствами Supabase через систему миграций. Каждая миграция представлена отдельным SQL-файлом с порядковым номером, что обеспечивает воспроизводимость схемы на любом окружении и облегчает отслеживание её эволюции.

Таблица 3.1 – Структура каталогов приложения

Путь	Назначение
app/(auth)/	Страницы регистрации и входа
app/(app)/	Защищённые страницы (каталог, профиль, клубы, встречи, админка)
app/api/	HTTP-маршруты (выгрузка CSV)
components/	Переиспользуемые компоненты интерфейса
lib/services/	Сервисный слой доступа к данным
lib/validators/	Схемы валидации Zod
lib/supabase/	Конфигурация клиентов Supabase
supabase/migrations/	SQL-миграции схемы базы данных

При создании пользователя через Supabase Auth автоматически срабатывает триггер, создающий соответствующую запись в таблице `profiles` с именем из метаданных регистрации:

```
CREATE FUNCTION public.handle_new_user() RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO public.profiles (user_id, name)
    VALUES (NEW.id, COALESCE(NEW.raw_user_meta_data->>'name', ''));
    RETURN NEW;
END $$ LANGUAGE plpgsql SECURITY DEFINER;
```

Такой подход гарантирует, что у каждого пользователя всегда существует профиль, и исключает рассогласование между таблицами `auth.users` и `profiles`.

3.3 Защита данных средствами Row Level Security

Главным механизмом разграничения доступа в приложении выступает Row Level Security (RLS) – средство PostgreSQL, позволяющее ограничивать видимость и модификацию строк таблицы на уровне самой базы данных. RLS применяется ко всем запросам независимо от того, из какой части приложения они сделаны, что исключает класс ошибок, связанных с забытой проверкой прав в коде сервера.

Для каждой таблицы определены отдельные политики на операции `SELECT`, `INSERT`, `UPDATE` и `DELETE`. Сводка ключевых политик представлена в

таблице 3.2.

Таблица 3.2 – Политики RLS для таблиц приложения

Таблица	Операция	Условие
profiles	SELECT	доступно всем
profiles	UPDATE	владелец профиля или администратор
clubs	SELECT	клуб активен или текущий пользователь – владелец
clubs	INSERT, UPDATE, DELETE	текущий пользователь – владелец клуба
meetings	SELECT	доступно всем
meetings	INSERT, UPDATE, DELETE	текущий пользователь – владелец клуба встречи
registrations	SELECT	собственная запись либо запись на встречу своего клуба
registrations	INSERT	запись создаётся от своего имени
registrations	DELETE	собственная запись

Расширенный доступ для администратора реализован отдельной политикой. Чтобы определить, является ли текущий пользователь администратором, требуется обратиться к его записи в таблице `profiles` и проверить значение поля `role`. Однако если такую проверку выполнить непосредственно в политике `profiles`, возникнет рекурсия: чтобы прочитать одну строку таблицы, СУБД сначала обращается к политике, а та, в свою очередь, читает другую строку той же таблицы. PostgreSQL обнаруживает этот цикл и прерывает запрос.

Для разрыва цикла проверка роли вынесена в отдельную хранимую функцию `public.is_admin()`, объявленную с признаком `SECURITY DEFINER`. Функции с таким признаком исполняются от имени их владельца и не активируют политики RLS при внутренних запросах. Политика `profiles` ссылается на эту функцию вместо прямого подзапроса, что устраняет рекурсию.

3.4 Аутентификация и роли пользователей

Аутентификация реализована средствами Supabase Auth. Регистрация и вход выполняются по паре «email – пароль»; для повторного использования

сессии Supabase устанавливает зашифрованный cookie. Серверные компоненты получают текущего пользователя вызовом `supabase.auth.getUser()`, который верифицирует сессию по cookie.

При регистрации в Server Action `registerAction` последовательно выполняются:

1. Валидация формы схемой `registerSchema` (Zod) – проверка формата email, минимальной длины пароля и совпадения поля подтверждения;
2. Вызов `supabase.auth.signUp` с передачей имени в `options.data.name` – его подхватит триггер создания профиля;
3. Детекция повторной регистрации по пустому массиву `identities` – Supabase не возвращает ошибку при дубликате email, поэтому критерий вычисляется вручную;
4. Перевод сообщения об ошибке Supabase на русский язык функцией `translateAuthError`.

Роли пользователей в системе устроены минимально. В таблице `profiles` хранится лишь два значения поля `role`: `member` и `admin`. Статус организатора, в отличие от роли, не сохраняется явно – он определяется наличием хотя бы одного клуба, в котором `owner_id = auth.uid()`. Такое решение позволяет любому участнику стать организатором без явного изменения роли и упрощает модель прав.

Блокировка пользователя выражается флагом `is_active = false` в таблице `profiles`. При следующем переходе по защищённой странице корневой компонент (`app`)/`layout.tsx` проверяет этот флаг и, в случае блокировки, инициирует выход из системы и редирект на страницу входа с параметром `?blocked=1`, по которому отображается соответствующее уведомление.

3.5 Сервисный слой и серверные действия

Все обращения к базе данных сосредоточены в модулях каталога `lib/services/`. Каждая функция возвращает значение типа `Result<T>`, представляющее собой алгебраический тип:

```
type Result<T> =
```

```
| { data: T; error: null }
| { data: null; error: string };
```

Такой контракт исключает выбрасывание исключений из сервисов и делает обработку ошибок явной в вызывающем коде. Сводка ключевых сервисов приведена в таблице 3.3.

Таблица 3.3 – Сервисный слой приложения

Модуль	Назначение
auth.ts	Регистрация, вход, выход, перевод ошибок Auth
profiles.ts	Чтение и обновление профиля, загрузка аватара
clubs.ts	CRUD клубов, получение клубов пользователя
meetings.ts	CRUD встреч, фильтрация каталога, выгрузка по клубу
registrations.ts	Запись и отмена записи, экспорт списка участников
admin.ts	Управление статусом пользователей и клубов

Мутации интерфейса выполняются через Server Actions. Действие получает данные формы, валидирует их, вызывает сервисный слой и при успехе перенаправляет на следующую страницу либо обновляет текущую вызовом `revalidatePath`.

3.6 Реализация ключевых модулей

3.6.1 Каталог встреч и фильтрация

Каталог встреч (страница `/meetings`) реализован как серверный компонент. Параметры фильтрации передаются через query-параметры URL и парсятся схемой `meetingFilterSchema`. Это позволяет сохранять состояние фильтра при обновлении страницы и делиться ссылкой на конкретную выборку.

Фильтр по уровню CEFR реализован как множественный выбор: пользователь может одновременно выбрать несколько уровней (например, B1 и B2), и результат включит встречи любого из них. Технически это реализовано через несколько вхождений параметра `cefr` в URL и оператор `IN` в запросе к базе данных.

Дополнительно каталог отсекает встречи, дата которых уже прошла (`date ≥ NOW()`), и встречи скрытых клубов через `inner-join` с фильтром

`club.is_active = true`. Внешний вид страницы каталога представлен на рисунке 3.1.

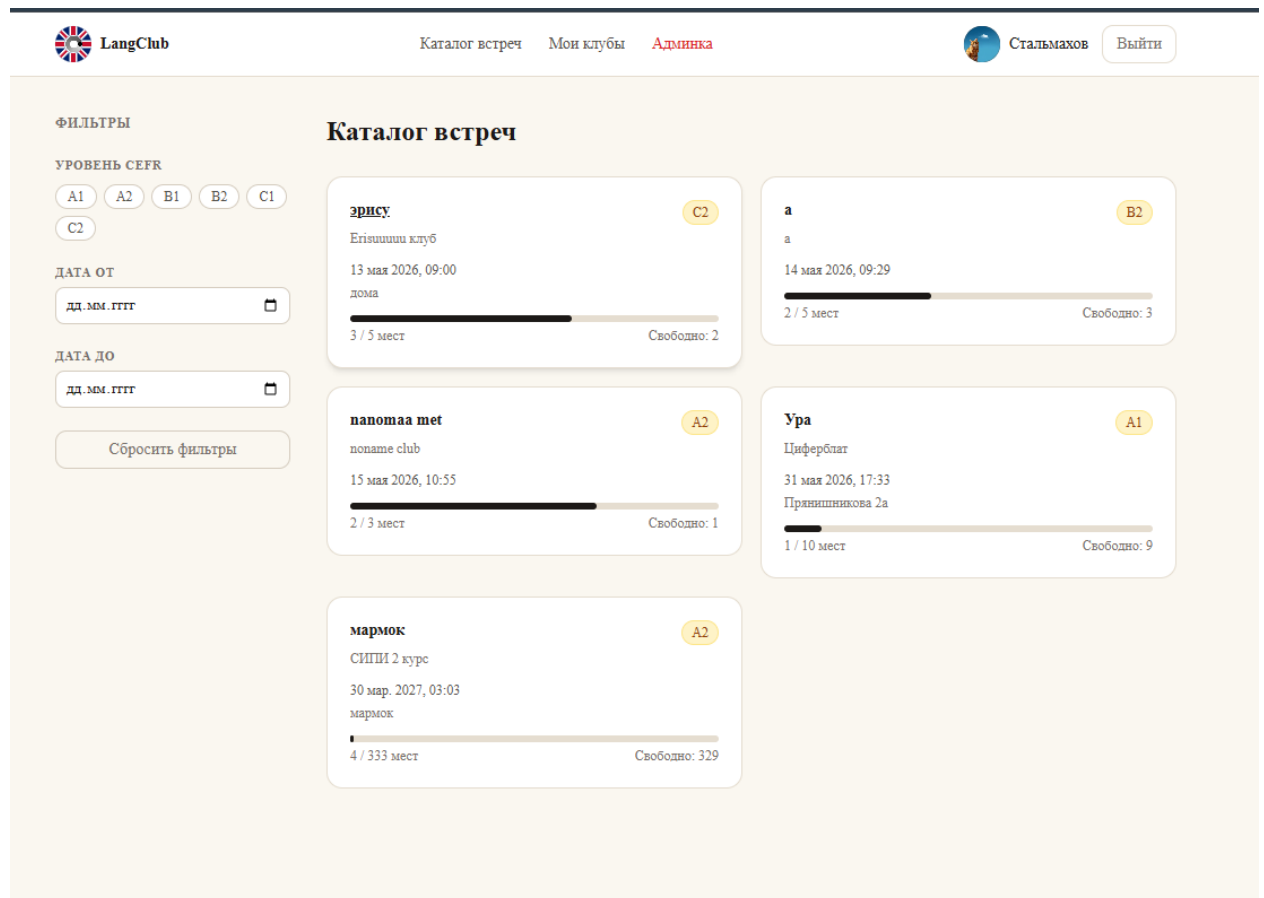


Рисунок 3.1 – Страница каталога встреч с фильтрами

3.6.2 Запись на встречу и защита от перепревышения

Реализация записи на встречу потребовала особого внимания к согласованности данных. Первоначальная версия считала количество занятых мест через агрегатный запрос `COUNT(*)` к таблице `registrations`, однако из-за RLS-политики, скрывающей чужие записи, обычный пользователь получал нулевой результат и мог записаться сверх лимита.

Решение состоит из двух частей.

Денормализация счётчика. В таблицу `meetings` добавлено поле `seats_taken`, инкрементируемое триггером `AFTER INSERT OR DELETE ON registrations`. Триггер выполняется в режиме `SECURITY DEFINER` и обходит RLS, что делает счётчик видимым всем пользователям и устраняет лишний запрос на чтение агрегата.

Атомарная проверка при вставке. Дополнительно создан триггер

BEFORE INSERT, который читает текущие `seats_total` и `seats_taken` с блокировкой строки (FOR UPDATE) и поднимает исключение, если мест не осталось. Этот уровень защиты исключает гонку при одновременной записи двух пользователей на последнее место.

```
CREATE FUNCTION check_seats_before_registration()
RETURNS TRIGGER SECURITY DEFINER AS $$
BEGIN
    SELECT seats_total, seats_taken INTO v_total, v_taken
    FROM meetings WHERE id = NEW.meeting_id FOR UPDATE;
    IF v_taken >= v_total THEN
        RAISE EXCEPTION 'seats_full';
    END IF;
    RETURN NEW;
END $$ LANGUAGE plpgsql;
```

Страница отдельной встречи с блоком записи представлена на рисунке 3.2.

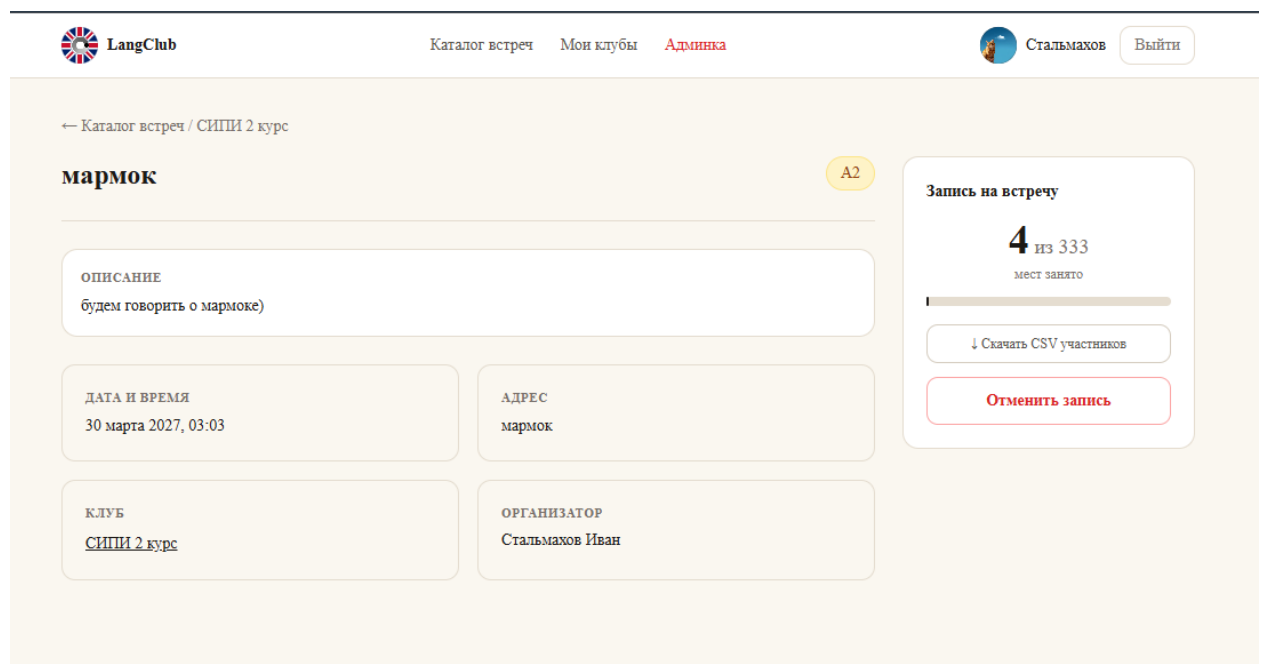


Рисунок 3.2 – Страница встречи с панелью записи

3.6.3 Управление клубами и встречами

Создание и редактирование клубов и встреч реализовано через формы с валидацией на стороне клиента (Zod + react-hook-form) и сервера (повторная

валидация в Server Action). Двойная валидация защищает от обхода клиентских проверок и обеспечивает целостность данных.

Принадлежность клуба определяется полем `owner_id`, автоматически предоставляемым в момент создания. Все операции изменения и удаления проверяют это поле и средствами RLS, и в коде сервиса.

Публичная страница клуба со списком предстоящих встреч представлена на рисунке 3.3, а панель организатора с управлением клубами и встречами – на рисунке 3.4.

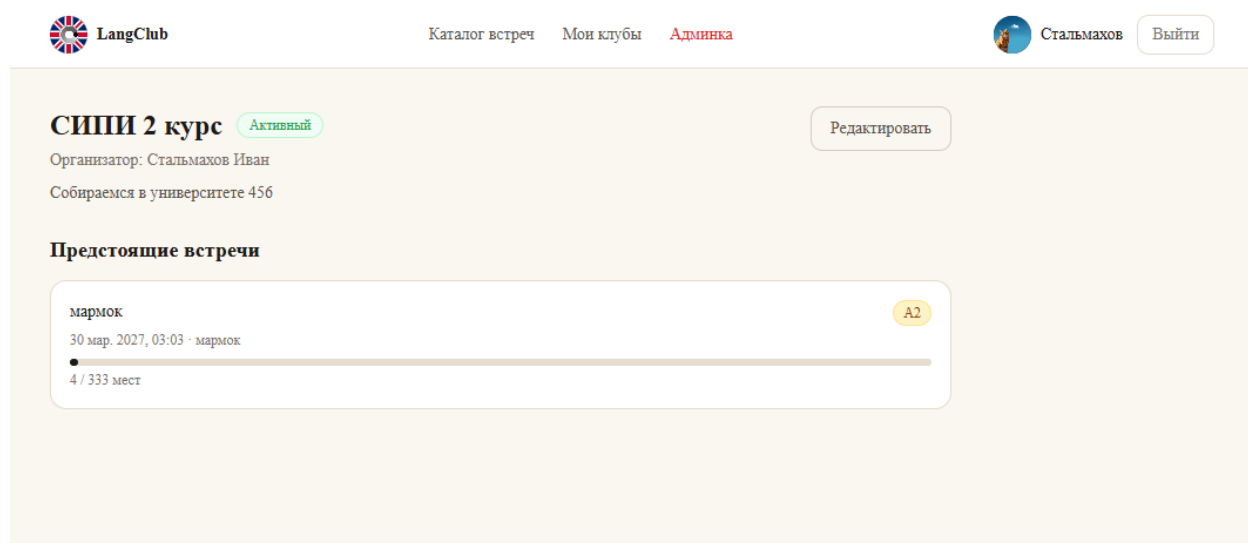


Рисунок 3.3 – Страница клуба с предстоящими встречами

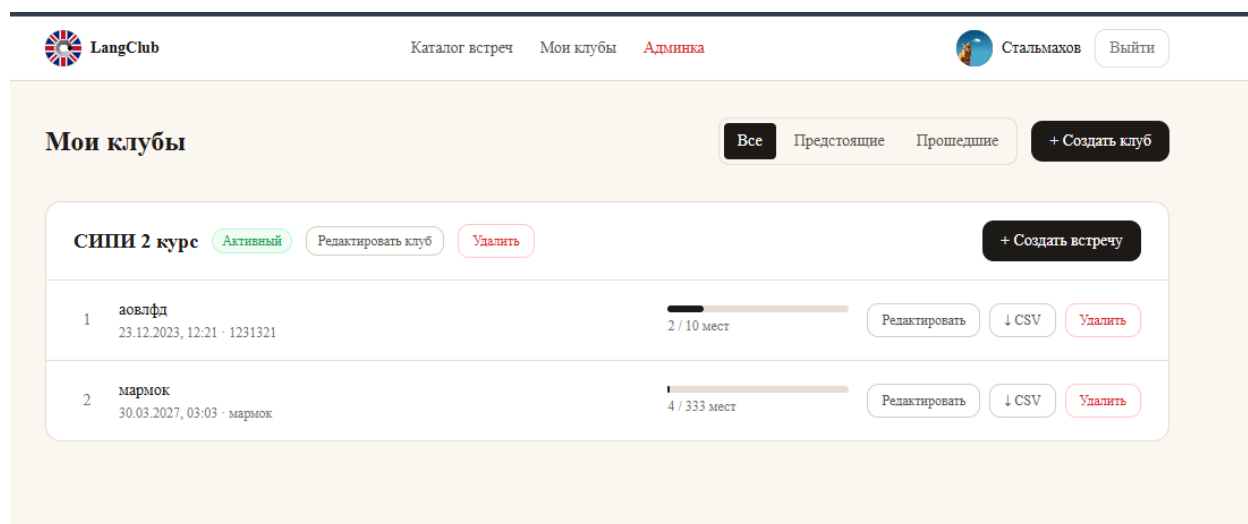


Рисунок 3.4 – Панель организатора со списком клубов и встреч

3.6.4 Загрузка аватара

Загрузка аватара реализована через Supabase Storage. На странице профиля файл выбирается через стандартный `<input type="file">`, после чего клиентский компонент `ProfileEditor` напрямую обращается к Supabase Storage из браузера, получает публичный URL и обновляет поле `avatar_url` в профиле. После записи вызывается `router.refresh()` для повторного рендера серверных компонентов (в том числе аватара в навигационной панели).

Доступ к бакету `avatars` ограничен политиками Storage: любой пользователь может читать файлы, но загружать или перезаписывать только в подпапку, имя которой совпадает с его `user_id`. Интерфейс редактирования профиля представлен на рисунке 3.5.

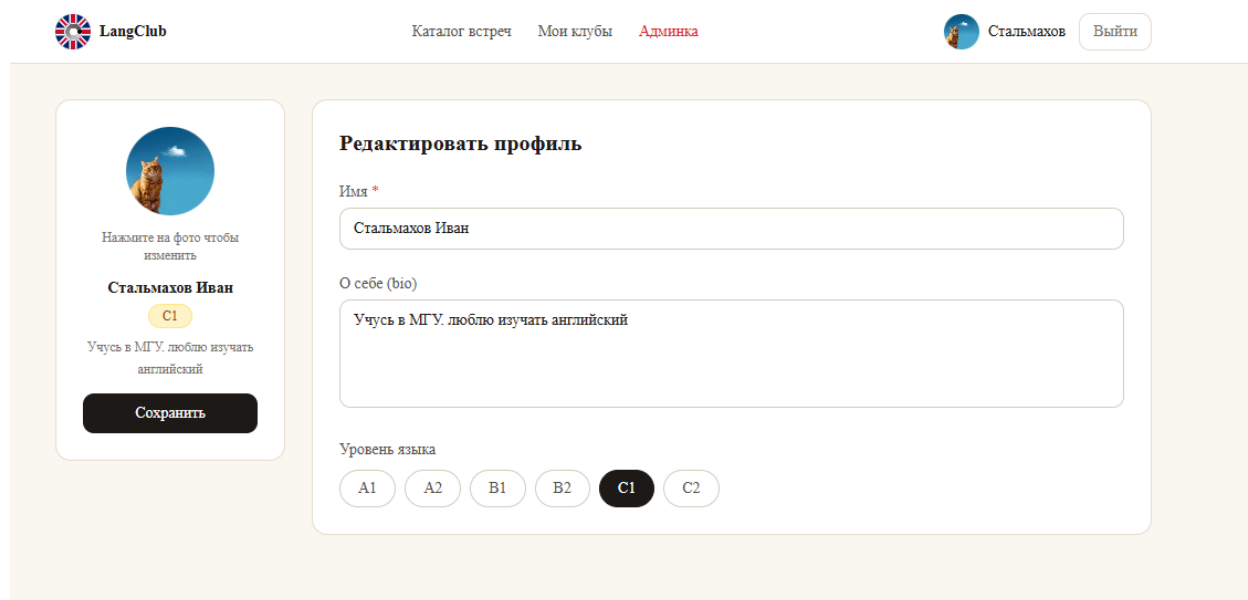


Рисунок 3.5 – Страница редактирования профиля с загрузкой аватара

3.6.5 Выгрузка списка участников в CSV

Список участников встречи доступен только владельцу клуба и выгружается по адресу `/api/meetings/[id]/export`. В отличие от прочих мутаций, реализованных через Server Actions, выгрузка файла потребовала Route Handler, поскольку требует возврата произвольного содержимого с заголовком `Content-Disposition`.

Список включает имя участника, уровень CEFR, дату записи и адрес электронной почты (получаемый через служебный метод `auth.admin.listUsers`). Файл формируется согласно RFC 4180 с использованием кодировки UTF-8 и

ВОМ для корректного отображения в Microsoft Excel.

3.6.6 Административная панель

Страница /admin доступна только пользователям с ролью admin. Корневой компонент маршрута проверяет роль и при её отсутствии выполняет редирект. Параллельно ссылка «Админка» в навигационной панели отображается только администраторам.

Панель состоит из двух разделов – управление пользователями и управление клубами. Для пользователей реализована блокировка и разблокировка с переключением флага `is_active`; администратор не может изменить статус собственной записи. Для клубов реализовано скрывание и возврат – скрытый клуб не отображается в каталоге, его встречи недоступны для записи, но владелец сохраняет доступ к редактированию. Интерфейс административной панели представлен на рисунке 3.6.

3.7 Тестирование

Для верификации ключевых компонентов написаны три типа тестов (таблица 3.4).

Таблица 3.4 – Покрывание тестами

Тип	Фреймворк	Покрываемая логика
Юнит	Vitest	Валидаторы Zod, форматтеры дат, генератор CSV
Интеграция	Vitest + моки	Сервис записи на встречу: ветки «занято», «дубликат», «клуб неактивен»
E2E	Playwright	Полный сценарий: регистрация, создание клуба и встречи, запись, выгрузка CSV

Особое внимание уделено сервису `registrations` как наиболее ответственному за согласованность данных. Тесты разработаны в стиле TDD до реализации сервиса, что позволило формализовать контракт функции через возвращаемый алгебраический тип `{ ok: true } | { ok: false; reason: ... }`.


LangClub

[Каталог встреч](#)
[Мои клубы](#)
[Админка](#)


Стальмахов
[Выйти](#)

Панель администратора

Пользователи

[Найти](#)

ИМЯ	ID	CEFR	РОЛЬ	СТАТУС	
Мармок Мармокович	29924972...	B1	member	Активен	Заблокировать
1	3107e760...	—	member	Активен	Заблокировать
	1a0e0a3c...	B2	member	Активен	Заблокировать
Erisuuuuu	1aac2a17...	A1	member	Активен	Заблокировать
Анненкова Дарья	a442fb86...	—	member	Активен	Заблокировать
Стальмахов Иван	45cbd6e3...	C1	admin	Активен	

Клубы

НАЗВАНИЕ	ВЛАДЕЛЕЦ (ID)	СТАТУС	
Циферблат	29924972...	Активен	Скрыть
no name club	1a0e0a3c...	Активен	Скрыть
a	3107e760...	Активен	Скрыть
Erisuuuuu клуб	1aac2a17...	Активен	Скрыть
СИПИ 2 курс	45cbd6e3...	Активен	Скрыть

Рисунок 3.6 – Панель администратора с разделами пользователей и клубов

3.8 Деплой приложения

Готовое приложение развёрнуто на платформе Vercel в режиме production. Деплой инициируется автоматически при пуше в основную ветку репозитория GitHub либо вручную командой `vercel -prod`.

Конфигурация подключения к Supabase передаётся через переменные окружения, заданные в настройках проекта Vercel. Среди них – URL проекта Supabase, публичный анонимный ключ для клиента и сервисный ключ для серверных операций (например, обращения к `auth.admin.listUsers` при выгрузке участников).

В результате реализованы все функциональные возможности, заявленные в главе 2: каталог встреч с фильтрацией, профиль пользователя с аватаром и уровнем CEFR, CRUD клубов и встреч, запись и отмена записи, выгрузка списка участников в CSV, административная панель с модерацией пользователей и

клубов.

ЗАКЛЮЧЕНИЕ

В результате проведённой работы подтверждена гипотеза о том, что специализированное веб-приложение, объединяющее каталог языковых клубов и инструменты записи на разговорные встречи, способно устранить информационную разрозненность, характерную для существующих универсальных платформ мероприятий и приложений языкового обмена. Теоретический анализ показал, что ни одно из распространённых решений (HelloTalk, Tandem, Meeturp) не сочетает одновременно групповой формат встреч, языковую специализацию и фильтрацию по уровню владения языком (CEFR), что обосновало необходимость разработки.

На основе полученных при разработке данных можно сформулировать следующие выводы:

1. Применение fullstack-фреймворка Next.js 15 в связке с облачной платформой Supabase обеспечивает архитектурную целостность приложения: серверная и клиентская части реализуются в едином репозитории, а аутентификация, реляционная база данных и файловое хранилище предоставляются единым поставщиком, что сокращает объём инфраструктурного кода без потери гибкости.
2. Защита данных средствами Row Level Security на уровне PostgreSQL является более надёжным механизмом разграничения доступа, чем проверки в коде сервера: правила применяются ко всем запросам независимо от точки входа, что исключает класс ошибок, связанных с пропущенными проверками прав.
3. Хранение счётчика занятых мест прямо в таблице встреч и проверка лимита средствами базы данных позволяют корректно обрабатывать запись на встречу даже тогда, когда несколько пользователей нажимают кнопку «Записаться» одновременно: превышение количества свободных мест становится невозможным.

Рекомендации и предложения. Для практического использования разработанной платформы в учебных заведениях и языковых центрах предлагается размещение на бесплатном тарифе Vercel и Supabase, что позволяет получить функционирующий продукт без затрат на инфраструктуру. Для роста

активности рекомендуется поэтапное наполнение каталога организаторами клубов и постепенное расширение модерации через административную панель.

Перспективы углубления темы. В дальнейшем развитии системы планируется внедрение функциональности уведомлений участников о приближающихся встречах по электронной почте и в Telegram, добавление оценок и отзывов на проведённые встречи, поддержка иных языков помимо английского, а также реализация мобильного клиента на базе общего API с веб-приложением.

При разработке пояснительной записки к курсовому проекту строго соблюдались требования, определяющие структуру и правила оформления научно-исследовательских работ.

СПИСОК ЛИТЕРАТУРЫ

- [1] Next.js Documentation: App Router [Electronic resource]. — [S. l. : s. n.]. — URL: <https://nextjs.org/docs> (online; accessed: 10.05.2026).
- [2] React Documentation: Quick Start [Electronic resource]. — [S. l. : s. n.]. — URL: <https://react.dev> (online; accessed: 27.04.2026).
- [3] Supabase Documentation: Database, Auth, Storage [Electronic resource]. — [S. l. : s. n.]. — URL: <https://supabase.com/docs> (online; accessed: 12.05.2026).
- [4] PostgreSQL Documentation: Row Security Policies [Electronic resource]. — [S. l. : s. n.]. — URL: <https://www.postgresql.org/docs/current/ddl-rowsecurity.html> (online; accessed: 12.05.2026).
- [5] TypeScript Handbook [Electronic resource]. — [S. l. : s. n.]. — URL: <https://www.typescriptlang.org/docs/handbook/intro.html> (online; accessed: 10.05.2026).
- [6] Tailwind CSS Documentation [Electronic resource]. — [S. l. : s. n.]. — URL: <https://tailwindcss.com/docs> (online; accessed: 10.05.2026).
- [7] Zod: TypeScript-first schema validation [Electronic resource]. — [S. l. : s. n.]. — URL: <https://zod.dev> (online; accessed: 10.05.2026).
- [8] HelloTalk – Language Exchange App [Electronic resource]. — [S. l. : s. n.]. — URL: <https://www.hellotalk.com> (online; accessed: 15.04.2026).
- [9] Tandem – Language Exchange [Electronic resource]. — [S. l. : s. n.]. — URL: <https://www.tandem.net> (online; accessed: 15.04.2026).
- [10] Meetup – Find Events and Groups [Electronic resource]. — [S. l. : s. n.]. — URL: <https://www.meetup.com> (online; accessed: 15.04.2026).
- [11] Vercel Documentation: Deployments [Electronic resource]. — [S. l. : s. n.]. — URL: <https://vercel.com/docs> (online; accessed: 12.05.2026).
- [12] ГОСТ 7.32-2017. Отчёт о научно-исследовательской работе. Структура и правила оформления [Текст]. — 2017.